

High Performance and Cloud Computing:
Final Project Report

Parallel Sample Sort Implementation &
Benchmarking

Francesco Bredariol
Data Science and Artificial Intelligence
University of Studies of Trieste
`francesco.bredariol@studenti.units.it`

September 26, 2026

1 Introduction

This report presents a hybrid MPI+OpenMP implementation in C language of the Parallel Sample Sort algorithm for sorting uniform distributed unsigned 64-bit integer data across multiple processes. I evaluate algorithmic choices including local sorting strategies, k -way merge mechanisms, and communication bottlenecks using `MPI_Alltoallv`. Furthermore, I conduct performance comparisons between native execution and containerized execution using Singularity, measuring strong and weak scaling, input skew robustness, and network injection overheads via OSU microbenchmarks. All the code implemented is available at: <https://github.com/frekkoz3/Parallel-Sample-Sort>.

1.1 Problem Description

The goal of this project is to sort a globally distributed array of N keys, evenly distributed across P MPI processes. Some assumptions have been made regarding the nature of the keys. In particular:

- **Data Distribution:** The data are uniformly distributed. This allows to achieve a theoretically optimal balance of the workload among processes. Some tests with skewed distributions have also been conducted.
- **Data type:** The keys are represented as **unsigned 64-bit integers** (`uint64_t`). This choice is particularly convenient for Radix Sort, since

unsigned integers can be processed directly through their binary representation without any transformation. Floating-point values could also be sorted using Radix Sort, but they require an additional transformation of their binary representation. In particular, the sign bit and the representation of negative values prevent the IEEE 754 representation from being directly ordered as an unsigned integer. Searching online I found [some techniques](#) proposed by *Terdiman and Herf* that transform floating-point values into integer keys that preserve their numerical ordering, allowing them to be processed by Radix Sort. Since handling floating-point keys was not strictly required for the scope of this project, unsigned integers were preferred in order to avoid this additional preprocessing step. Another interesting thing that motivate the choice of 64 bits is the size of the datasets considered. With up to 10^9 uniformly distributed keys, a 32-bit key provides only $2^{32} \approx 4.3 \times 10^9$ possible values. This means that duplicate keys are expected. A 64-bit key provides 2^{64} possible values, which makes collisions extremely unlikely under a uniform distribution. The larger key space therefore allows the experiments to be performed with large datasets while keeping the number of duplicate keys negligible.

1.2 Overall Procedure

The implementation combines OpenMP for shared-memory thread-level parallelism and MPI for inter-process communication. The overall procedure is the following:

1. **Local Generation:** Each process generates its local chunk of keys using a slightly different, but deterministic, seed.
2. **Local Sort:** Each process sorts its local chunk using OpenMP parallelization.
3. **Regular Sampling:** Each process selects $P - 1$ regularly spaced samples from its sorted chunk.
4. **Global Pivot Selection:** All samples are gathered using `MPI_Allgather`, sorted, and used to select $P - 1$ global pivots.
5. **Local Partitioning:** Each process partitions its local data according to the pivots into P buckets.
6. **Global All-to-All Exchange:** Each process exchanges buckets with every other process (`MPI_Alltoall` for the sizes, followed by `MPI_Alltoallv` for the data).
7. **Final k -Way Merge:** Each process performs a k -way merge of the P incoming sorted streams. The streams are already sorted because

the local chunks were sorted in Step 2 and the buckets preserve their internal ordering.

Notice that, under the regular distribution assumptions, no process ever holds the entire dataset. Moreover, the final sorted array does not exist in the typical sense: it can be obtained by concatenating the local chunks stored on each process in rank order.

1.3 Deliverables

In addition to the code and the experimental results (including plots, run specifications, and comments), the following aspects are reported:

1. Hardware Stack
2. Software Stack
3. Compilation Flags
4. CLI Arguments
5. Statistical Treatment of Results

1.3.1 Hardware

All experiments were executed on the GENOA partition of the Orfeo cluster by Area Science Park. The output of the `lscpu` command on the **Genoa-006** node is shown below:

Variable	Value
Architecture	x86_64
CPU Model	AMD EPYC 9374F (Genoa, Zen 4)
Sockets	2
Cores per Socket	32
Threads per Core	1
CPU op-modes	32-bit, 64-bit
Byte Order	Little Endian
L1d cache	2 MiB (64 instances)
L1i cache	2 MiB (64 instances)
L2 cache	64 MiB (64 instances)
L3 cache	512 MiB (16 instances)

Table 1: `lscpu` command output on the **Genoa-006** node

According to the [official documentation of the Orfeo cluster by Area Science Park](#), the two sockets are connected through an **Infinity Fabric** consisting of three links at 16 GT/s, providing a theoretical bandwidth of 96 GB/s.

1.3.2 Software Stack

As required, the project uses both OpenMP and OpenMPI. The complete software stack used for the project is reported below.

Variable	Value
MPI	Open MPI 5.0.5
MPI compiler	mpicc
Underlying compiler	GCC 14.3.1
OpenMP	OpenMP 4.5
OpenMP runtime	GCC libgomp
Singularity	4.3.1
OSU	7.4

Table 2: Software stack

However, for containerized execution, the Ubuntu 24.04 image requires MPI version 4.1.6 (which is an available module on Orfeo).

1.3.3 Compilation Flags

The code is compiled using **mpicc** with the following flags:

Type of Flag	Values
C standard	C11
Optimization	-O3
Warnings	-Wall -Wextra -pedantic
Parallelism	-fopenmp

Table 3: Compilation flags

1.3.4 CLI Arguments

The code provides a Command Line Interface with several arguments. All available arguments are reported below.

1.3.5 Experimental Statistical Robustness

Statistical robustness is an important aspect of HPC benchmarking. For this project, a simple “5 runs” strategy was used. The results are reported as the average over five runs, together with a visual indication of the variance.

This approach was selected to limit the number of requests to the computational nodes while collecting enough samples to reduce the impact of large outliers.

Argument	Possible Values
<code>--n</code>	Number of keys to sort (default: 10^6)
<code>--oversample</code>	Regular samples per virtual rank multiplier (default: 1)
<code>--seed</code>	Random seed for generated inputs (default: 1)
<code>--distribution</code>	<code>uniform</code> , <code>skewed</code> , <code>few-unique</code>
<code>--sorting</code>	<code>quick</code> , <code>radix</code>
<code>--radix_bits</code>	Digit size for radix sort (default: 8)
<code>--merging_strat</code>	<code>basic</code> , <code>bin</code> , <code>heap</code>
<code>--where_save</code>	Path to the output file (default: <code>./results/results.csv</code>)
<code>--print-limit</code>	Number of first sorted keys to print (default: 0)
<code>--help</code>	Displays the help message

Table 4: CLI arguments

1.3.6 Profiling

Profiling was performed directly in the code, without external profilers, using the `MPI_Wtime()` function to measure the execution time of each relevant subroutine.

Since the application is an MPI parallel job, the execution times measured by the different processes are reduced on rank 0, and the maximum value is selected. This is because the execution time of a parallel operation is determined by its slowest process.

In particular, the following subroutines were profiled:

1. Keys Generation
2. Global Signature Computation
3. Local Sort
4. Global Sampling
5. Local Partitioning
6. Local Merging
7. Global Sort Verification
8. Global Signature Verification
9. Inter-Node Communication
10. Total Time

Experiment	Fixed Parameter	What varies
Strong scaling – native	$N = 10^9$, uniform	$P \in \{4, 8, 16, 32, 64\}$
Strong scaling – container	same	same
Weak scaling – native	$N/P = 2 \times 10^7$, uniform	$P \in \{4, 8, 16, 32\}$
Weak scaling – container	same	same
Skewed Input – native	$N = 10^8$, Zipf $\alpha = 1.1$	$P \in \{8, 16, 32\}$
Skewed Input – container	same	same
OSU <code>osu_alltoallv</code>	2–64 ranks, intra + inter node	Message size: 1 KiB–64 MiB

Table 5: Requested experimental configurations

1.3.7 Experiments List

The requested experiments are reported below. They will be described in more detail in the following sections.

Additional experiments were also performed and will be described directly in the corresponding sections. Their names are:

1. Local Sort Comparison
2. Quick Sort Pivot Strategy Study
3. Radix Sort Optimal Digit Size Study
4. Merging Strategy Comparison
5. Optimal Oversampling Factor Study
6. MPI `Alltoallv` Bottleneck and Refactoring into Point-to-Point Communication

2 Implementation Details

The final implementation is composed of the Sample Sort main function, which has already been described, and the individual components called by the main function. These are:

1. Parallel Verifier
2. Local Sorting Algorithm
3. Bucket Merging Strategy

In this section, it is describe in detail how these components are implemented.

2.1 Parallel Verifier

The parallel verifier performs two main checks:

1. **Global sorting verification**
2. **Data loss verification**

2.1.1 Global sorting verification

Since the data is distributed across ranks, global sorting is verified in two steps:

1. **Local sorting:** verify that each rank's local bucket is sorted.
2. **Boundary condition:** verify that the last key of rank r is less than or equal to the first key of rank $r + 1$.

For local sorting, the verifier scans consecutive pairs of keys and records the index of the first violation. If no violation is found, `first_bad` is set to `nkeys`, which is larger than any valid index. The check is parallelized with an OpenMP `parallel for` and a minimum reduction on `first_bad`, so that the smallest invalid index is preserved.

The result is then converted into `local_err`, which is 1 if the local bucket is unsorted and 0 otherwise. The local index is also shifted by the rank's global offset, producing a global index. If no local error is found, the index is set to `options.nkeys + 1`, ensuring that it cannot interfere with the subsequent minimum reduction.

To verify the boundaries between ranks, each rank sends the first key of its local output to the previous rank. At the same time, it receives the first key from the next rank. Non-blocking `MPI_Isend` and `MPI_Irecv`, followed by `MPI_Waitall`, are used to perform these communications without unnecessary serialization.

Each rank can then check whether $output_{last}^{(r)} > output_{first}^{(r+1)}$.

If so, a boundary error is detected and the corresponding global index is stored in `bad_index`.

Finally, MPI reductions combine the results from all ranks. A sum reduction on `local_err` and `boundary_err` determines whether any error occurred, while a minimum reduction on `bad_index` identifies the first global position at which the sorting property is violated. Therefore, rank 0 can determine whether the complete distributed output is globally sorted and, if not, report the first invalid index.

2.1.2 Data loss verification

The second verification step checks whether the output contains the same elements as the input. This is done using an order-independent signature based on two quantities: the sum and the bitwise XOR of a mixed version of each key.

For each local array, the signature is computed with an OpenMP parallel loop:

$$\begin{aligned}\text{sum} &= \sum_i \text{mix_key}(\text{keys}[i]), \\ \text{xor} &= \bigoplus_i \text{mix_key}(\text{keys}[i]).\end{aligned}$$

Both operations are associative and commutative, so they can be safely computed in parallel using OpenMP reductions. Since the signature is independent of the order of the elements, it can be used to detect changes in the set of keys, although it does not formally prove that the output is a permutation of the input.

After computing the local input signature, the values are combined across ranks. A sum reduction is used for the `sum` component, while a bitwise XOR reduction is used for the `xor_value` component. This produces a single global signature on rank 0.

The same procedure is then applied to the sorted output. The resulting global output signature is compared with the input signature using `same_signature`. If both the sum and XOR values match, `signature_ok` is set to 1; otherwise, it is set to 0.

This provides a cheap diagnostic for detecting data loss or modification during the sorting process.

2.2 Local Sort

The choice of the local sorting algorithm is one of the main factors affecting the overall performance of Parallel Sample Sort. I considered two algorithms, Quick Sort and Radix Sort, and evaluated several implementation and tuning choices for both. The results of these experiments are discussed in the following sections.

2.2.1 Quick Sort

Quick Sort recursively partitions the input array around a pivot. At each step, elements smaller than the pivot are placed in the left partition, while larger elements are placed in the right partition. The two partitions are then sorted recursively. The quality of the pivot is therefore crucial, as an unbalanced partition can significantly increase the running time. The optimal pivot split in two partition of the same size (the pivot would be the median).

Three-way partitioning The standard two-way partitioning was replaced with a three-way scheme, which divides the input into

$$[\text{begin}, lt) < \text{pivot}, \quad [lt, gt) = \text{pivot}, \quad [gt, \text{end}) > \text{pivot}.$$

This is particularly useful when the input contains many duplicate keys, since all elements equal to the pivot are handled in a single step and do not need to be processed again by subsequent recursive calls.

Pivot selection I evaluated three pivot selection strategies:

- **In-the-middle (ITM):** selects the element at the center of the current range.
- **Median-in-the-middle (MITM):** computes the median of the three elements around the center of the range.
- **Tukey’s ninther:** divides the range into three groups of three elements, computes the median of each group, and then takes the median of the three resulting values.

The ITM strategy has negligible selection cost, but its performance depends strongly on the distribution of the input. The ninther generally provides a better pivot estimate, but requires more comparisons. In the beginning also a Median Of Median (MoM) strategy was experimented, but it was revealed to be too expensive. In my experiments, MITM provided the best trade-off: it is only slightly more expensive than ITM while producing more balanced partitions. It was therefore used as the default strategy in the final implementation.

Parallelization The parallel version uses OpenMP tasks to recursively sort the two partitions. Instead of creating two new tasks after every partitioning step, the left partition is assigned to a new OpenMP task while the current thread immediately processes the right partition. This reduces task-management overhead while still exposing parallelism.

For sufficiently small partitions, parallelization is stopped and the remaining range is sorted serially. I use a serial cutoff of 25,000 elements. Below this threshold, the overhead of creating OpenMP tasks is not justified.

Finally, the serial Quick Sort uses insertion sort for very small partitions. With an insertion-sort cutoff of 8 elements, the recursion overhead is avoided where insertion sort is more efficient.

2.2.2 Radix Sort

Radix Sort is a non-comparison-based sorting algorithm that processes the keys digit by digit. Since the keys are unsigned 64-bit integers, their binary representation can be divided into a fixed number of digits. This fixed number of digit will be called simply **digits** from now on. Starting from the least significant digit, the elements are stably sorted according to each digit. After processing all digits, the array is sorted.

The inner sorting operation is a counting sort. For each digit, a histogram is built by counting how many elements belong to each possible bucket. Prefix sums (which can be seen, in a probabilistic stand point of the counting sort, as discrete unnormalized empirical cumulative distribution functions) are then used to determine the starting position of each bucket, after which the elements are scattered into the destination array. A ping-pong technique is also use: two buffers are alternated between consecutive passes, avoiding unnecessary element-wise copying. Since counting sort is stable, the ordering established by previous digits is preserved.

Digit size The number of bits processed at each pass represents a trade-off between the number of passes and the size of the histograms. Larger digits reduce the number of passes, but require more buckets and therefore larger histograms.

I evaluated different digit sizes and restricted the implementation to values that are exact divisors of the key size. Since the keys are 64-bit integers, the number of bits processed at each pass must divide 64. This avoids the need for special handling of a final partial digit and results in a simpler implementation. The final digit size of 8 was selected experimentally based on the execution time of the local sorting phase.

Parallelization The histogram construction and scattering phases can be parallelized, while the computation of the global bucket positions is performed serially. Each thread processes a separate portion of the input and builds its own local histogram. These histograms are then combined to determine the global starting position of every bucket.

I tried to keep the histograms small enough to fit in the cache, and I used `posix_memalign` to align them in memory, with the aim of reducing false sharing between threads.

To allow threads to scatter their elements independently, each thread computes a private offset for every bucket. The offset is obtained by adding the global starting position of the bucket to the number of elements belonging to the same bucket that are assigned to preceding threads. Consequently, different threads write to disjoint portions of each bucket.

This offset scheme also preserves the stability of counting sort: within each thread, elements are processed in their original order, while the thread

offsets determine their relative position inside the global bucket. After each digit is processed, the input and scratch arrays are swapped and the next digit is processed. This ping-pong scheme avoids unnecessary copies between passes.

For small input ranges, the parallel implementation falls back to the serial version. This avoids the overhead of creating and synchronizing OpenMP threads when the amount of work is too small to benefit from parallelism.

2.3 Merging Strategy

The final step implementation detail left to discuss is the merging of the sorted streams belonging to each destination bucket. After the data exchange, each process receive several sorted chunks, one from each source process (even if they can indeed be empty). In the end these chunks must be merged into a single sorted sequence.

I implemented and compared three different k -way merging strategies: a basic iterative approach, a binary pairwise merge, and a direct heap-based merge.

2.3.1 Basic

The basic implementation directly scans the heads of all k input streams to find the smallest available key. After selecting the minimum, the corresponding stream is advanced and the process is repeated until all elements have been consumed.

This approach is simple and requires no additional data structure, but finding the minimum requires examining k stream heads for every output element. For n total elements, the resulting complexity is

$$O(nk).$$

Of course this is not asymptotically optimal, but it provides a simple baseline against which the other merging strategies can be evaluated. Also, I will see how in small number of ranks regime this can be the optimal choice.

2.3.2 Binary

The binary strategy reduces the number of active streams by repeatedly merging pairs (from this the name "binary") of streams. In the first round, streams $(0, 1)$, $(2, 3)$, and so on are merged. The resulting streams are then merged again in pairs, and the process continues until only one stream remains.

Each round processes every element exactly once, while the number of rounds is

$$O(\log k).$$

Therefore, the overall complexity is

$$O(n \log k).$$

The implementation uses a ping-pong strategy based on two buffers. After each round, the source and destination buffers are swapped. If the number of streams is odd, the last stream is copied directly to the destination buffer and participates in the following round.

This approach reduces the number of comparisons compared with the basic method and uses the same standard two-way merge operation at every step.

2.3.3 Heap

The heap-based implementation performs a direct k -way merge using a min-heap. Initially, the first element of every non-empty stream is inserted into the heap. The root therefore contains the smallest currently available element across all streams.

After extracting the minimum, the next element from the same stream is inserted into the heap. This process is repeated until all streams have been exhausted.

Since the heap contains at most k elements, each extraction and insertion requires

$$O(\log k)$$

time. Therefore, for n total elements, the overall complexity is

$$O(n \log k).$$

Compared with the binary strategy, the heap performs the merge in a single pass over the input streams, but every output element requires heap update. The binary strategy instead performs several complete passes over the data, with each pass using a simple two-way merge.

3 Experimental Results

This section presents the first part of the experimental evaluation. Rather than focusing on scalability, the goal of these experiments is to identify the parameters and strategies that provide the best overall performance for the implementation.

For each experiment, I first describe the objective and the expected behavior, then report the experimental configuration, present the results, and discuss the observed behavior.

3.1 Local Sort Comparison

As discussed previously, two local sorting algorithms have been implemented: Quick Sort and Radix Sort. The first set of experiments focuses on comparing their performance and determining the most suitable algorithm for the final implementation.

Two main aspects were investigated:

1. the effect of the key size (32 or 64 bits)
2. the absolute performance of Quick Sort and Radix Sort

I expected the key size to have a significant impact on Radix Sort, since its running time depends directly on the number of bits that must be processed. In contrast, I expected the key size to have little or no direct impact on Quick Sort, since comparisons between integer keys have essentially the same structure for both 32-bit and 64-bit values.

Since this experiment focuses exclusively on the local sorting phase, the tests were performed at the OpenMP level. The complete program was executed using a single MPI rank, while the number of OpenMP threads was varied from 1 to 64, doubling at each step.

The common configuration used for all experiments was:

1. `nkeys = 5 · 107`
2. `nrank = 1`
3. `keys_distribution = uniform`

Bit Size Comparison The first experiment evaluates the effect of the key size on the two sorting algorithms.

Figure 1 shows the results for Quick Sort. As expected, the key size has only a minor effect on the sorting time. The differences between the 32-bit and 64-bit configurations are small and are not consistent across all thread counts. For example, at four threads the 64-bit configuration is faster than the 32-bit configuration. This suggests that, for Quick Sort, the additional width of a 64-bit key does not represent a significant performance bottleneck for this workload.

The behavior is considerably different for Radix Sort, as shown in Figure 2. The 64-bit configuration is consistently slower than the 32-bit one.

This behavior is expected because Radix Sort processes the key bits in multiple passes. Assuming the same digit size, a 64-bit key requires twice as many digit passes as a 32-bit key. However, the measured slowdown is not necessarily exactly 2×.

I therefore retain 64-bit keys for the final implementation because of the considerations regarding the number of possible duplicate values discussed

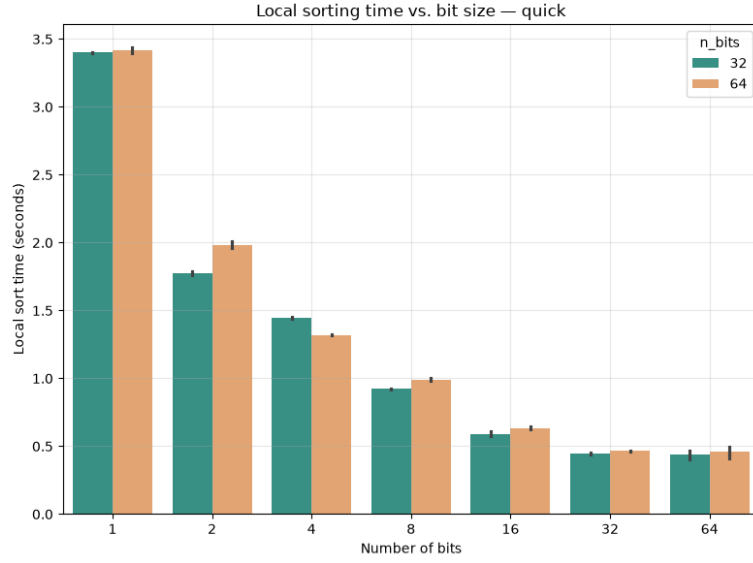


Figure 1: Quick Sort sorting time as a function of the number of OpenMP threads for 32-bit and 64-bit keys.

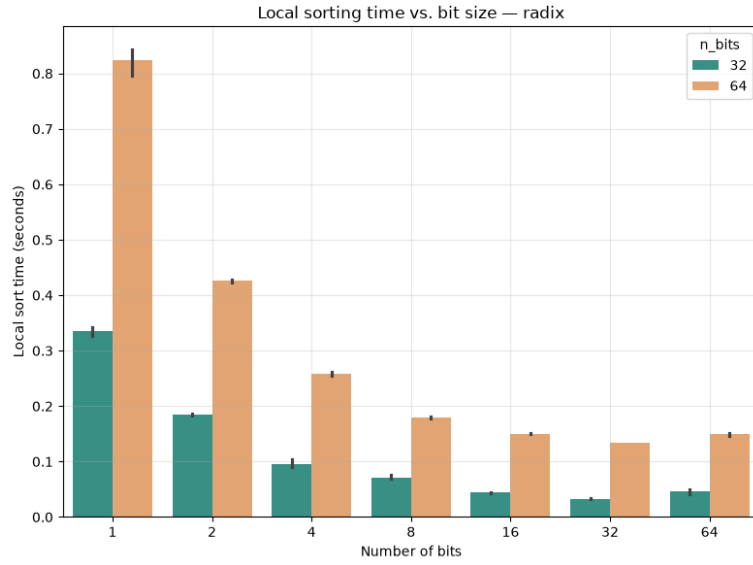


Figure 2: Radix Sort sorting time as a function of the number of OpenMP threads for 32-bit and 64-bit keys.

in the problem description. Nevertheless, both key sizes are considered in the comparison of the two sorting algorithms, allowing their absolute performance to be evaluated under both configurations.

Sort Comparison The second experiment directly compares the absolute performance of Quick Sort and Radix Sort. The comparison is performed separately for 32-bit and 64-bit keys in order to highlight once again both the difference between the algorithms and the effect of the key size.

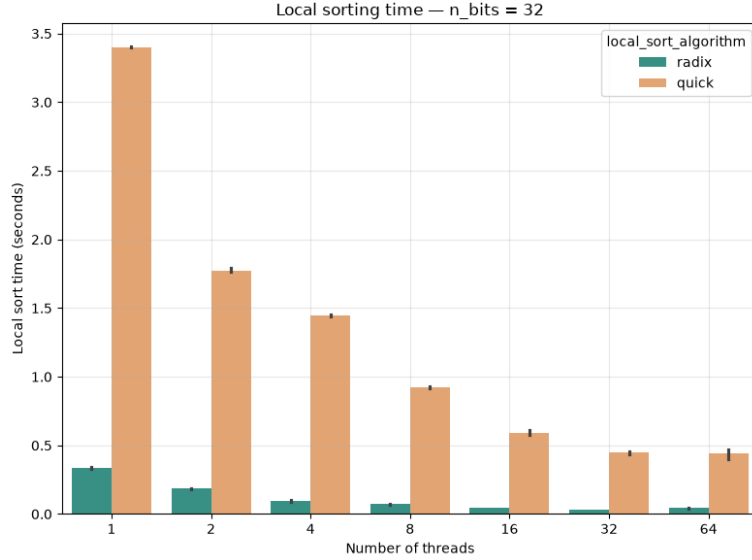


Figure 3: Sorting time as a function of the number of OpenMP threads for 32-bit keys.

In both configurations, Radix Sort consistently outperforms Quick Sort. The difference is particularly evident for this workload, which is well suited to Radix Sort: the keys are fixed-width unsigned integers, allowing the algorithm to process their binary representation directly without any additional transformation.

Furthermore, the OpenMP implementation of Radix Sort exposes parallelism in a relatively structured way. Histogram construction and the subsequent scatter operation can be divided among threads, while the per-thread offsets ensure that threads write to disjoint regions of each bucket. This results in a more controlled parallel implementation than the task-based recursive parallelism used by Quick Sort.

The results therefore support the choice of Radix Sort as the default local sorting algorithm from now on. The 64-bit configuration is retained as the default key representation, while the 32-bit results are useful for understanding the impact of key width on the sorting phase.

3.2 Merging Strategy Comparison

The merging strategy is an important component of Parallel Sample Sort, since its performance can become increasingly relevant as the number of MPI

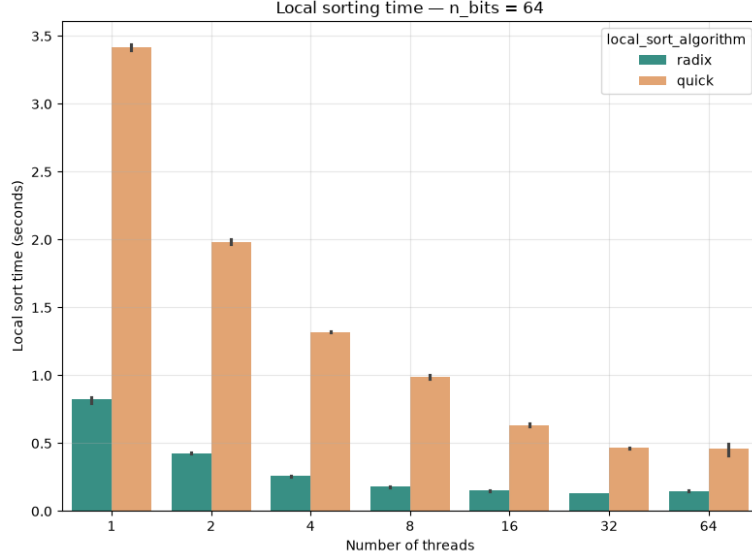


Figure 4: Sorting time as a function of the number of OpenMP threads for 64-bit keys.

ranks grows. As described previously, three strategies were implemented: basic, binary, and heap.

Letting n be the total number of elements to merge and k the number of sorted input streams, I remember that the theoretical computational costs of the three strategies are:

- **Basic:** $O(nk)$
- **Binary:** $O(n \log k)$
- **Heap:** $O(n \log k)$

Although the binary and heap strategies have the same asymptotic complexity, they differ in their constant factors and implementation overhead.

Since the number of input streams is exactly the number of MPI ranks, I expect the merging strategy to become increasingly important as the number of ranks increases. For a small number of ranks, the basic strategy may perform well due to its simplicity and low overhead. As the number of ranks increases, however, its $O(nk)$ complexity should make it increasingly less competitive, while the binary and heap strategies should benefit from their $O(n \log k)$ complexity.

The experiments were performed using a single OpenMP thread, while the number of MPI ranks was varied. The complete program was executed with a fixed number of keys and a uniform key distribution.

The common configuration used for all experiments was:

1. `nkeys =  $5 \cdot 10^7$`
2. `n_threads = 1`
3. `keys_distribution = uniform`

Notice that under uniform distribution I can expect an average bucket size of $\frac{1}{\#ranks} 5 \cdot 10^7$.

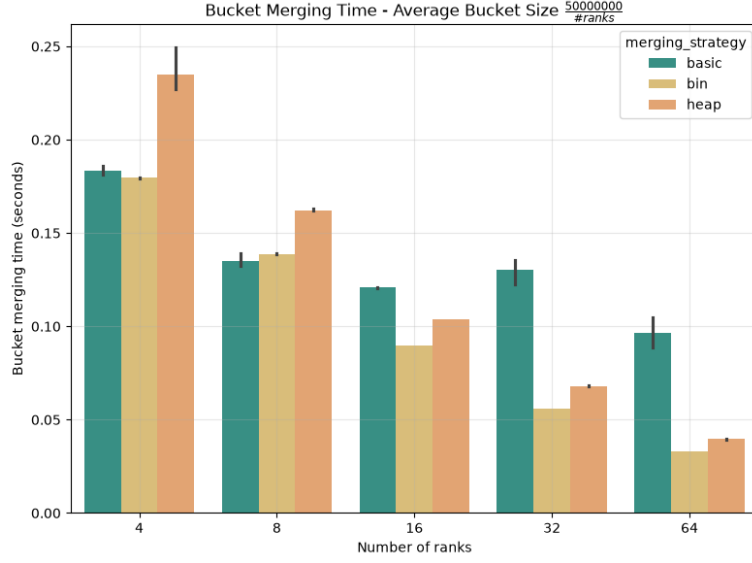


Figure 5: Bucket merging time as a function of the number of MPI ranks for the three merging strategies.

Figure 5 shows the measured merging time for the three strategies as the number of MPI ranks increases. The results are consistent with the expected theoretical behavior. For a small number of ranks, the basic strategy performs competitively, as the number of streams is small and its low overhead compensates for its less favorable asymptotic complexity.

As the number of MPI ranks increases, the performance of the basic strategy deteriorates more rapidly than that of the other two strategies. This is consistent with its $O(nk)$ complexity. The binary and heap strategies instead show better behavior as the number of ranks increases, in agreement with their $O(n \log k)$ complexity.

The binary strategy is consistently slightly faster than the heap strategy. This can be probably be attributed to the lower overhead of pairwise merging compared with the continuous maintenance of the heap structure, despite the two approaches having the same asymptotic complexity.

Based on these results, the binary merging strategy is selected as the default strategy from now on.

3.3 Oversample Factor Tuning

The oversample factor controls the number of samples collected by each MPI process during the sample selection phase. In my implementation, the number of samples collected by each process is determined by

$$n_{\text{samples per rank}} = \text{oversample_factor} \cdot n_{\text{ranks}}.$$

Consequently, with P MPI ranks and an oversample factor f , the total number of samples is

$$n_{\text{samples}} = fP^2.$$

For an oversample factor of 1, the total number of samples is therefore P^2 .

The purpose of oversampling is to obtain a more representative approximation of the global key distribution when selecting the pivots. Increasing the oversample factor provides more samples from which to select the pivots and can therefore lead to better balanced buckets. However, increasing it also increases the amount of data involved in the sample collection and pivot selection phases, introducing additional communication and processing overhead.

The optimal value is therefore expected to be a trade-off between the quality of the selected pivots and the cost of obtaining and processing additional samples.

To study this trade-off, two experiments were performed using different numbers of MPI ranks: 8 and 64. The other parameters were kept fixed as follows:

- `nkeys`: 5×10^7
- `nranks`: 8 or 64
- `nthreads`: 8 for 8 ranks and 1 for 64 ranks
- `keys_distribution` = `uniform`

Two metrics were considered: the total execution time and the communication time.

For 8 MPI ranks, the effect of the oversample factor on the total execution time is relatively small, making it difficult to identify a clear optimum from the total-time measurements alone. This is expected to some extent, since the overall amount of data and the number of communicating processes are relatively small.

The results become more pronounced with 64 MPI ranks. In this case, the choice of the oversample factor has a clearer impact on the total execution time. In particular, increasing the oversample factor initially improves

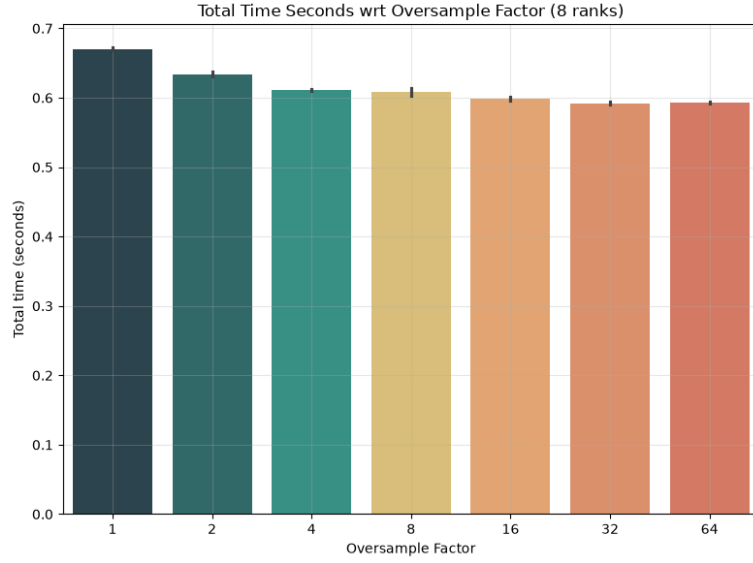


Figure 6: Total execution time as a function of the oversample factor with 8 MPI ranks.

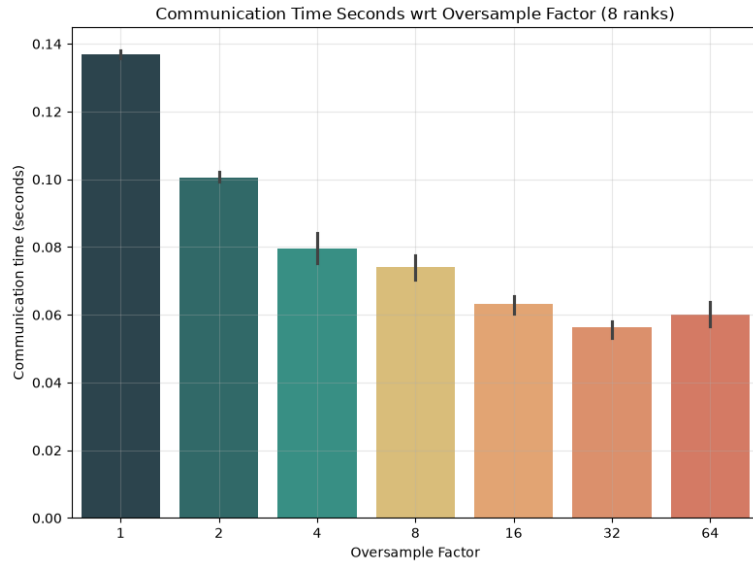


Figure 7: Communication time as a function of the oversample factor with 8 MPI ranks.

the behavior, while values that are too large introduce additional overhead without providing a corresponding benefit.

Interestingly, the communication-time measurements show a similar trend for both configurations. The best trade-off is reached around an oversample

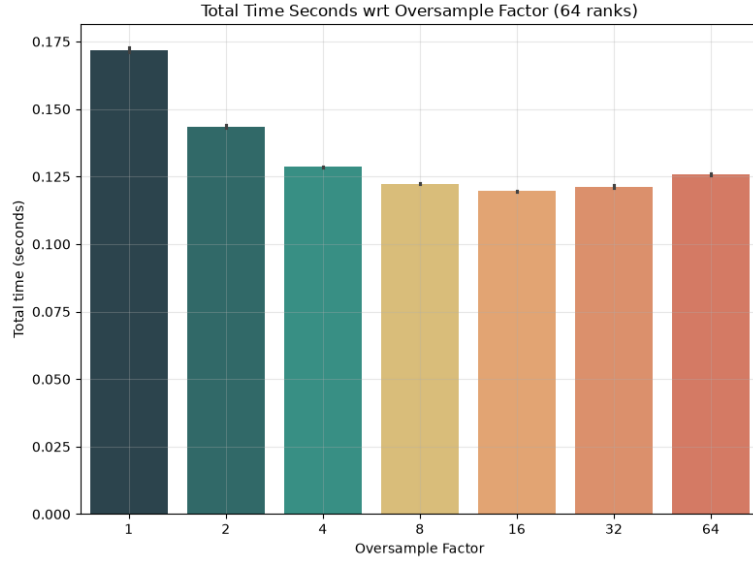


Figure 8: Total execution time as a function of the oversample factor with 64 MPI ranks.

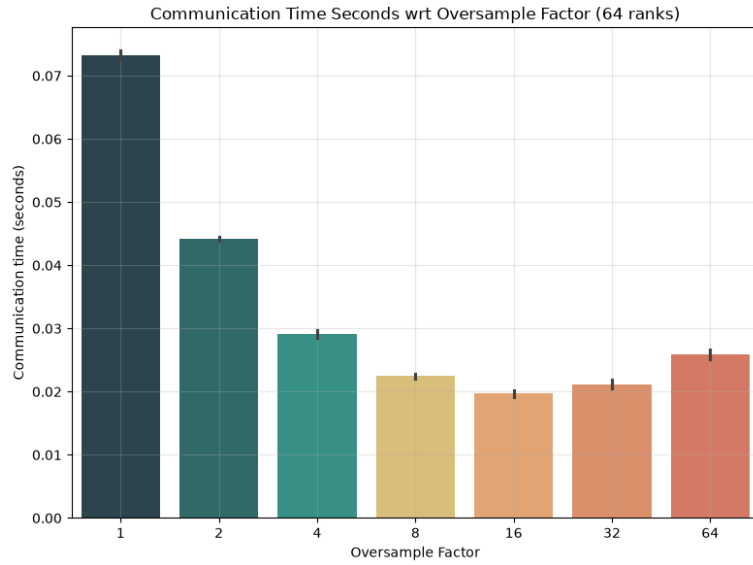


Figure 9: Communication time as a function of the oversample factor with 64 MPI ranks.

factor of 32. This suggests that, for the tested configurations, increasing the number of samples beyond this point does not provide enough additional benefit to compensate for the increased communication and processing cost.

In terms of the total number of samples, an oversample factor of 32

corresponds to

$$n_{\text{samples}} = 32P^2.$$

This means that, in the range explored by these experiments, the optimal configuration corresponds to a total number of samples between the basic P^2 case and the P^3 scale. The experiments therefore suggest that moderate oversampling provides a good balance between obtaining representative pivots and limiting the overhead of the sampling phase.

Anyway, since the experiment was done after the other ones, and since I didn't have high priority to the Orfeo queue, all the scaling experiment has an oversample factor of 1.

3.4 Point-to-Point Communication Comparison

The main bottleneck of the communication is the `MPI_Alltoallv` to exchange the data belonging to the different destination processes. As proposed in the assignment I evaluate the performance impact of replacing this collective communication with an explicit point-to-point implementation based on `MPI_Sendrecv`.

The optimized `MPI_Alltoallv` implementation should exploit the communication capabilities of the MPI library to optimize the exchange between processes. On the other hand, the point-to-point implementation requires a total of $O(P^2)$ potential communication pairs, where P is the number of MPI ranks. This lead me to expect the point-to-point (ptp) approach to become increasingly expensive as the number of ranks grows. However, I also expect this difference to become clearly visible only for sufficiently large numbers of MPI processes and maybe particularly unbalanced data.

The experiment was performed using a modified version (commented in the source code) of the implementation, where the `MPI_Alltoallv` communication was replaced by the point-to-point implementation. The experimental configuration presents a number of MPI ranks increasing exponentially from 4 to 256.

The common configuration used for all experiments was:

- `nkeys`: 10^9
- `nthreads`: 1
- `keys_distribution` = uniform

The two main metrics considered are the communication time, `time_communication_seconds`, and the total execution time, `total_seconds`. The former allows me to isolate the effect of the communication implementation, while the latter captures its impact on the complete execution.

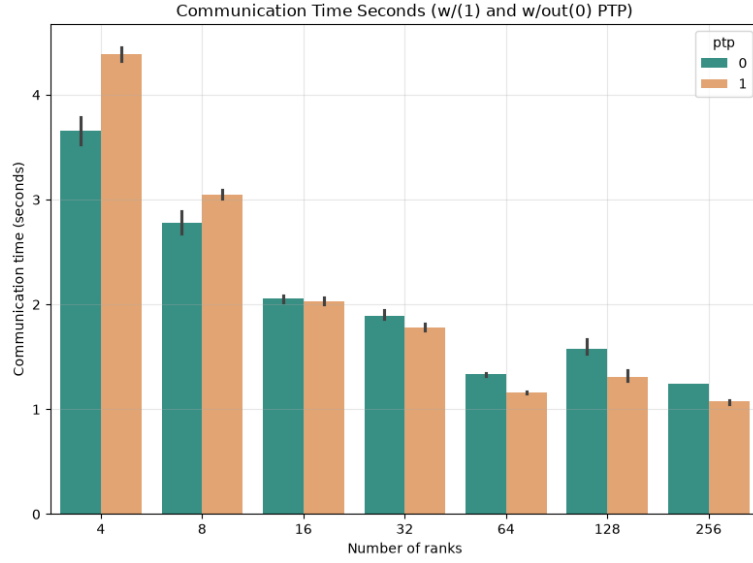


Figure 10: Communication time as a function of the number of MPI ranks for `MPI_Alltoallv` and point-to-point communication.

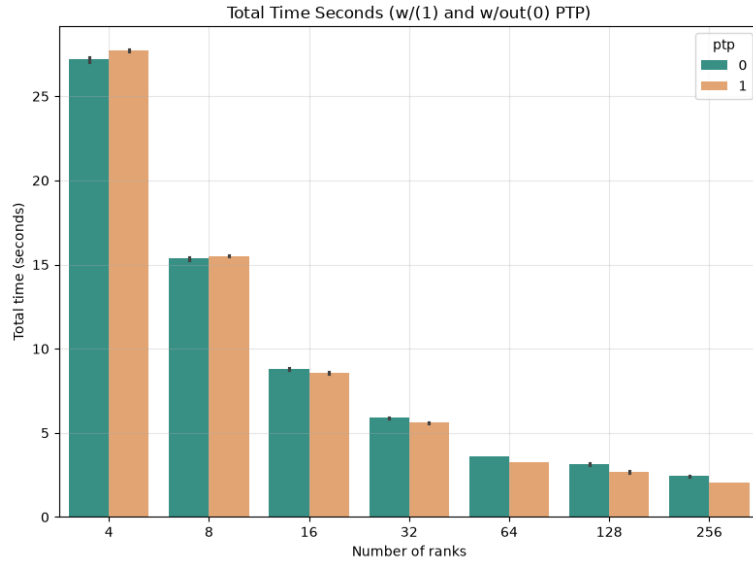


Figure 11: Total execution time as a function of the number of MPI ranks for `MPI_Alltoallv` and point-to-point communication.

Contrary to the initial hypothesis, the difference between the two communication approaches is not clearly visible in the tested range. Even as the number of MPI ranks increases to 256, the point-to-point implementation does not show the expected significant deterioration. In fact, in these

experiments it appears to be slightly faster than `MPI_Alltoallv`, both in terms of communication time and total execution time.

This result does not absolutely mean that the point-to-point approach is generally more efficient than `MPI_Alltoallv`. My personal idea about this particular case is that this is a particularly lucky scenario for the point-to-point approach that penalizes the little optimization overhead of `MPI_Alltoallv`.

So the experiment does not reveal a substantial performance difference between the two approaches under the tested conditions. The optimized `MPI_Alltoallv` implementation is retained for the final version, since it provides the standard collective abstraction for the all-to-all exchange.

4 Scaling Laws Results and Skewed Distribution

I can now move to the analysis of the scaling behavior of the implementation. The experiments are first conducted on the native implementation, while the effects of containerization are analyzed separately in a later section.

As requested from the assignment, two different scaling models are considered: strong scaling and weak scaling.

In addition a stress test on the procedure has been conducted, using skewed keys distributions instead of the usual uniform.

4.1 Strong Scaling

In **strong scaling**, the total problem size is kept fixed while the number of workers increases. The objective is to determine how much the execution time can be reduced by providing additional computational resources.

A nice example found somewhere while studying is a single house that has to be built. Increasing the number of workers can reduce the construction time, since more people can work simultaneously. However, as the number of workers becomes large, there is less work available for each worker and more coordination is required. Strong scaling experiments help me understand at what point communication and synchronization overheads can overcome the useful computation, limiting the achievable speedup. This behavior is described by Amdahl's law

The experiment focuses primarily on the MPI level, since the number of workers corresponds directly to the number of MPI ranks. The number of OpenMP threads is fixed to one, while the number of MPI ranks is increased exponentially from 4 to 256. The common configuration used across all runs is:

- `nbits`: 64
- `nkeys`: 10^9

- **nthreads:** 1
- **keys_distribution:** uniform
- **sorting_strategy:** radix
- **merging_strategy:** binary

For strong scaling, parallel efficiency is defined as

$$E_N = \frac{S_N}{N} = \frac{T_1}{NT_N},$$

where T_1 is the runtime on a single processor, T_N is the runtime using N processors, and S_N is the corresponding speedup.

In this experiment, however, no measurement with a single MPI rank was performed. Therefore, T_4 is used as the baseline, and the reported efficiency is computed as

$$E_N^{(4)} = \frac{T_4}{NT_N}.$$

This provides an efficiency relative to the four-rank configuration rather than to a single processor. Consequently, to express the values as an approximation of the conventional efficiency relative to one processor, the measured values are multiplied by 4.

$$E_N \approx 4E_N^{(4)}.$$

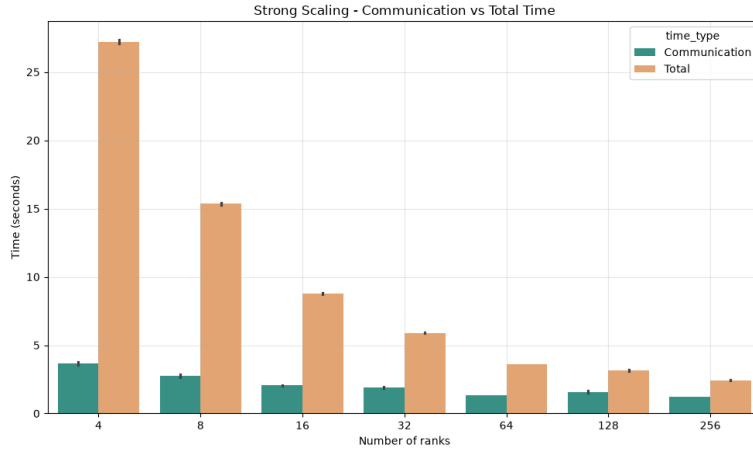


Figure 12: Total execution time and communication time as a function of the number of MPI ranks.

As shown in Figure 12, the total execution time decreases as the number of MPI ranks increases. The communication time also decreases in absolute

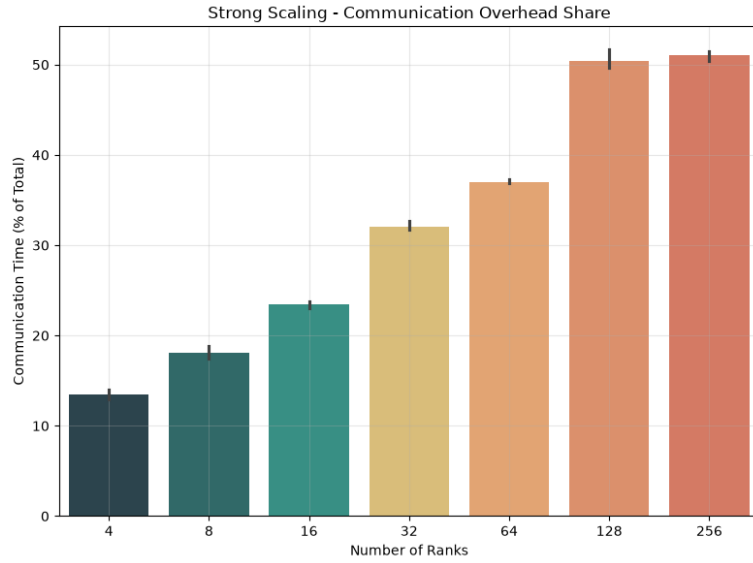


Figure 13: Communication time as a percentage of total execution time for different numbers of MPI ranks.

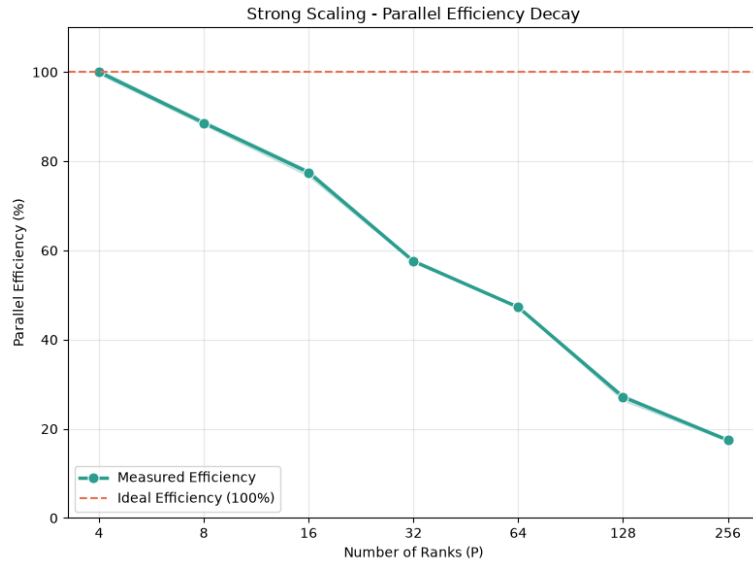


Figure 14: Parallel efficiency as the number of MPI ranks increases.

terms, although at a slower rate than the total runtime. This becomes more evident when considering the relative contribution of communication: as shown in Figure 13, the communication share increases from slightly above 10% at low rank counts to more than 50% at the highest number of ranks.

This increasing communication fraction is also reflected in the parallel

efficiency shown in Figure 14. As the number of MPI ranks increases, the efficiency progressively decreases, reaching approximately 20% at the highest tested rank count. This indicates that, despite the reduction in absolute execution time, the benefit of adding more MPI ranks becomes progressively smaller because communication and synchronization represent an increasingly significant portion of the total execution time. In fact I could say that the sweet spot, for the proposed configuration, could have been reached at 64 ranks.

4.2 Weak Scaling

In **weak scaling**, instead, the amount of work assigned to each worker remains approximately constant while the total problem size increases with the number of workers. The objective is to determine whether the execution time can be maintained approximately constant as the system grows.

This time is like considering an increasing number of workers, with each worker responsible for building an additional part of a larger house. As the number of workers increases, the total amount of work also increases. One would think that the additional workers compensate for the additional workload, keeping the execution time stable. In reality, however, the workers must communicate and synchronize with each other, and this overhead generally increases with the size of the system. Weak scaling experiments therefore allows me to observe when the communication and synchronization costs associated with increasing the number of MPI ranks become significant compared with the actual computational work. This behavior is related to Gustafson's law.

Again the experiment focuses primarily on the MPI level. The number of openMP threads is fixed to four, while the problem size N is increased proportionally to the number of MPI ranks. In particular, the amount of work per rank is kept constant according to

$$\frac{N}{P} = 2 \times 10^7,$$

where

$$P$$

is the number of MPI ranks. The common configuration used across all runs is:

- **nbits:** 64
- **nthreads:** 4
- **keys_distribution:** uniform
- **sorting_strategy:** radix

- **merging_strategy**: binary

For weak scaling, the parallel efficiency is computed relative to the four-rank configuration. Let T_4 be the average total runtime measured with four MPI ranks. The efficiency is therefore defined as

$$E_P \approx \frac{T_4}{T_P} \times 100,$$

where T_P is the total runtime measured with P MPI ranks. Unlike strong scaling, the problem size increases with the number of ranks, so an ideal weakly scaling implementation would maintain approximately constant execution time.

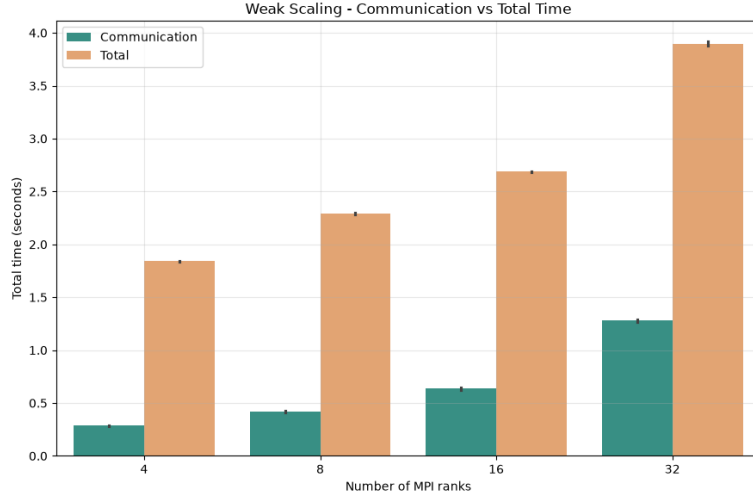


Figure 15: Total execution time and communication time as a function of the number of MPI ranks in the weak scaling experiment.

In the weak scaling experiment, the total problem size increases proportionally with the number of MPI ranks, while the amount of work assigned to each rank remains constant. As shown in Figure 15, both the total execution time and the absolute communication time increase as the number of ranks grows. This indicates that the additional parallel resources do not completely compensate for the increased global problem size and communication requirements.

The communication overhead becomes increasingly significant as the system scales. Its share of the total execution time increases from approximately 15% at the lowest rank count to around 33% at the highest one, as shown in Figure 16. Thus, although the computational workload per rank remains approximately constant, the communication cost grows with the size of the parallel system.

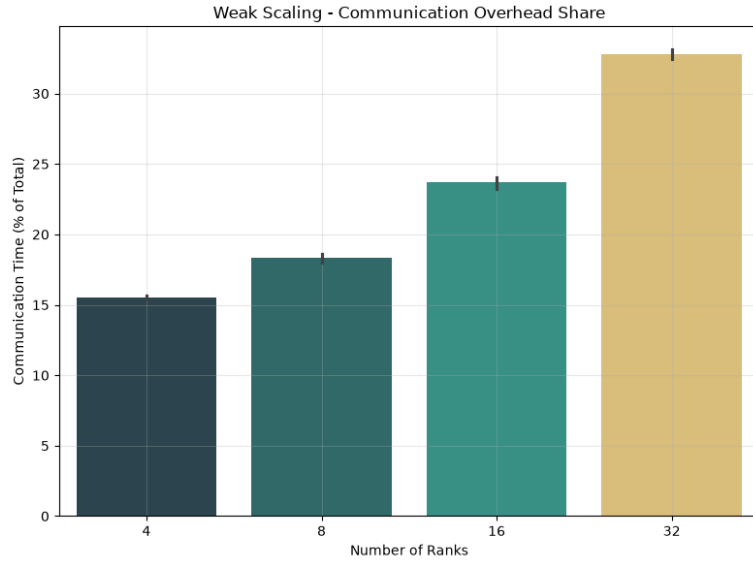


Figure 16: Communication time as a percentage of total execution time for different numbers of MPI ranks in the weak scaling experiment.

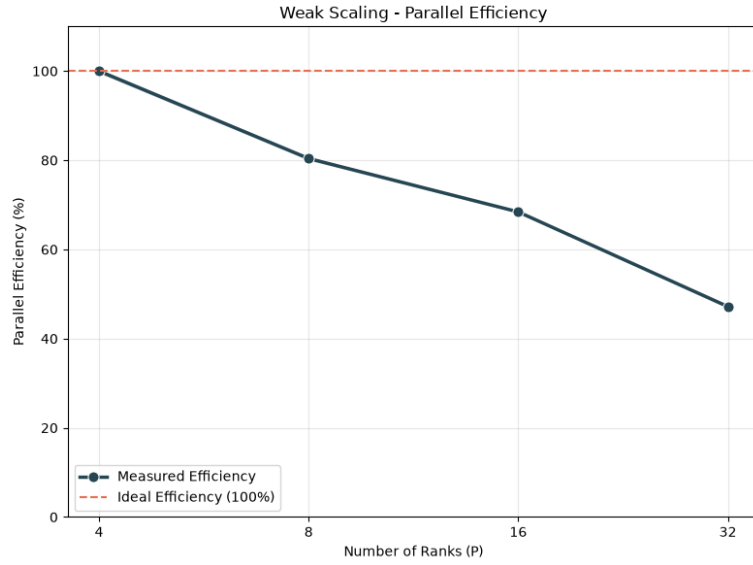


Figure 17: Parallel efficiency as the number of MPI ranks increases in the weak scaling experiment.

This behavior is also reflected in the parallel efficiency shown in Figure 17. Using the four-rank configuration as the baseline, the efficiency progressively decreases as more MPI ranks are introduced, reaching approximately 45% at the highest tested rank count. Therefore, the implementa-

tion exhibits a noticeable loss of weak-scaling efficiency, primarily associated with the increasing communication overhead as the number of MPI ranks grows.

4.3 Skewed Distribution Stress Test

Skewed key distributions represent a challenging case for Parallel Sample Sort. The main issue is that a non-uniform distribution can lead to unbalanced final buckets. If a large fraction of the keys falls into a small portion of the key space, some processes may receive significantly more data than others. As already mentioned, since the overall execution time is ultimately determined by the slowest process, such load imbalance can negatively affect the performance of the parallel algorithm.

In this experiment, I evaluate the behavior of the implementation under two non-uniform distributions that were already implemented in the serial baseline: **skewed** and **few-unique**. They are compared with the uniform distribution used in the previous experiments.

The **few-unique** distribution generates keys uniformly from a very small set of only 1024 possible values. Consequently, many keys are identical:

$$k_i \in [0, 1023].$$

This represents an extreme case with a very high number of duplicate keys.

The **skewed** distribution instead combines a concentrated region with a wider uniform component. Approximately 90% of the generated keys are selected from a small range of size

$$\frac{n_{\text{keys}}}{16} + 1,$$

while the remaining keys are generated over the full 64-bit range. This distribution tries to mimic the Zipf distribution with alpha 1.1.

The experiments use the following configuration:

- **n_keys**: 10^9
- **n_ranks**: 16
- **n_threads**: 4
- **merging_strategy**: binary
- **sorting_algorithm**: radix

The main metrics considered are the communication time and the total execution time. The merging time is also reported to investigate how the different distributions affect the final merging phase.

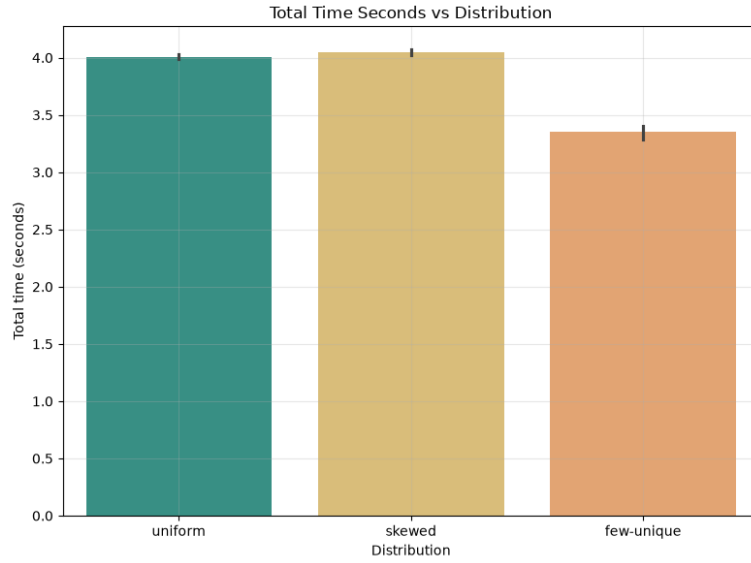


Figure 18: Total execution time for the different key distributions.

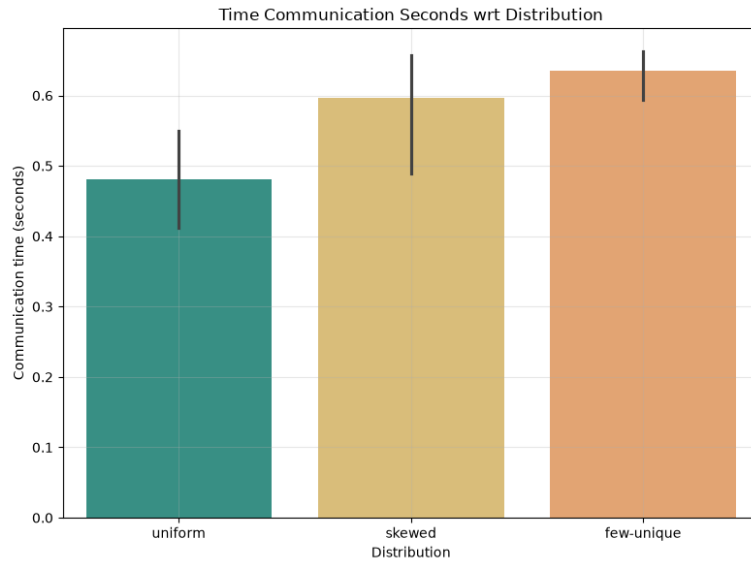


Figure 19: Communication time for the different key distributions.

Fig19 show exactly the expected behavior for the different distributions: the uniform distribution produces the most balanced buckets and for that achieves the lowest communication time. On the other side, the skewed and few-unique distributions produce a less balanced distribution of data among the destination processes, which led to a higher communication costs.

Nonetheless this behavior is not directly reflected in the total execution

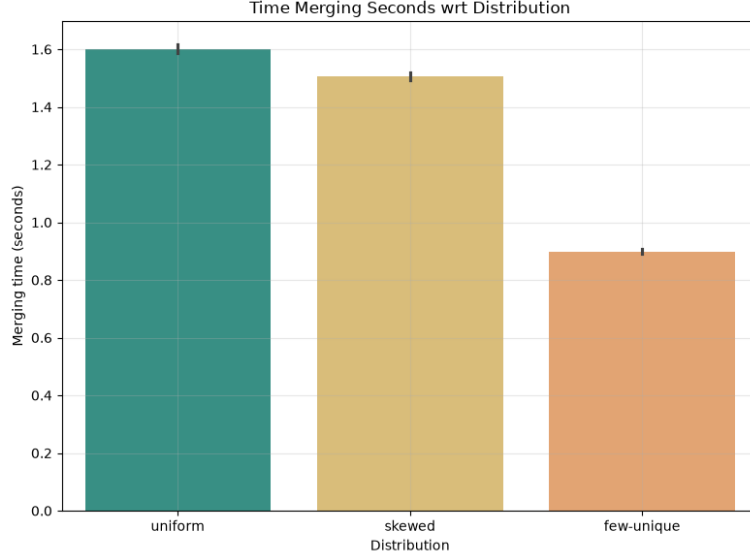


Figure 20: Merging time for the different key distributions.

time, this can be seen in Fig18. In particular, both the few-unique and skewed distribution achieved a lower total execution time than the uniform distribution, despite requiring more communication.

Investigating why is that, looking at Fig20, I found out that the merging time is the actual relevant factor. As a consequence of the non-uniform distribution, a significant number of destination buckets can remain empty. These empty buckets correspond to empty input streams in the merging phase and therefore require little or no actual merging work. As a result, the merging phase can be substantially shorter for the non-uniform distributions.

This result should not be interpreted as evidence that Parallel Sample Sort is generally better suited to non-uniform distributions. Rather, it highlights an interesting trade-off between communication and computation. In the tested configuration and with the current implementation, the additional communication overhead caused by the unbalanced buckets was more than compensated by the reduction in merging time resulting from the presence of empty buckets. This illustrates how the different phases of the algorithm can interact in non-trivial ways, making the overall performance dependent not only on load balance, but also on the amount of effective work performed in each phase.

5 Containerization

The containerized version of the application follows the workflow

Docker image $\rightarrow$ Singularity image $\rightarrow$ execution on the HPC cluster.

The Docker image contains the application together with its compiler, MPI, NUMA, InfiniBand, and UCX dependencies. The resulting image is converted to Singularity format and executed on the cluster using the host's Slurm environment.

5.0.1 NUMA topology

As a first verification, the visibility of the host NUMA topology from inside the Singularity container was examined. Inspection of

`/sys/devices/system/node/`

confirmed that the complete NUMA topology of the AMD EPYC Genoa compute nodes remains visible inside the container. In particular, all eight NUMA domains, from `node0` to `node7`, were accessible.

This is important because the application explicitly relies on NUMA-aware memory allocation and thread placement. Functions such as `numa_alloc_onnode()` can therefore still address the host NUMA domains from inside the container. Similarly, CPU and thread affinity mechanisms remain available to the application.

Consequently, containerization does not appear to introduce an abstraction layer for the NUMA topology itself: the application can still interact with the physical NUMA architecture of the underlying node.

The containerized implementation was then compared with the native implementation using the same experimental configurations. Three experiments were considered: strong scaling, weak scaling, and the distribution workload.

5.0.2 Strong Scaling

For the strong-scaling experiment, the same configuration used for the native execution was applied to the containerized version. The total execution times are shown in Figure 21.

The results show that the total execution time of the containerized implementation is very similar to that of the native implementation. No significant performance degradation is observed in this experiment.

This result indicates that, for the strong-scaling workload, the computational and memory-related parts of the application are largely unaffected by the container boundary. In particular, the previously verified NUMA topology remains accessible, allowing the same NUMA-aware allocation and thread-placement mechanisms to be used.

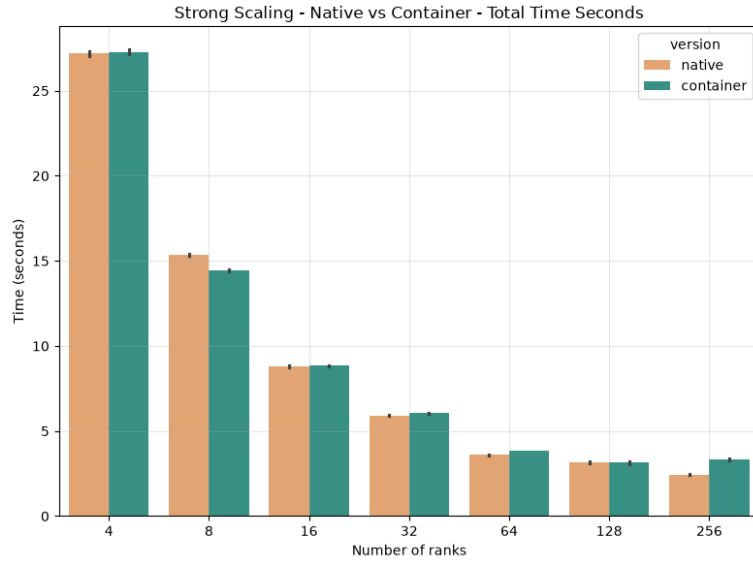


Figure 21: Strong-scaling total execution time for the native and containerized implementations.

5.0.3 Weak Scaling

The weak-scaling experiment was performed using the same configuration as the native experiment. The total execution time is presented in Figure 22.

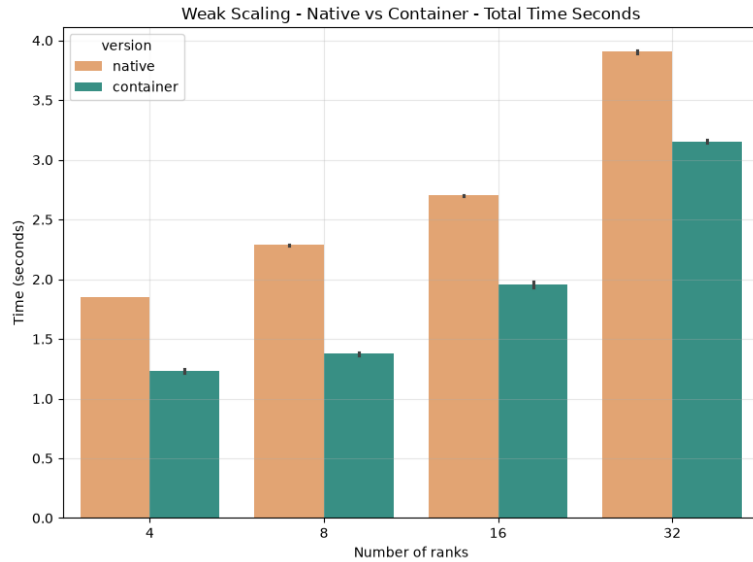


Figure 22: Weak-scaling total execution time for the native and containerized implementations.

Interestingly, the containerized implementation shows a lower total execution time than the native implementation for this experiment. This result was unexpected, since containerization alone should not provide a systematic computational advantage over the native execution, particularly when both executions use the same hardware and experimental configuration.

To investigate this observation, the execution was decomposed into its principal components. Figure 23 shows the local-sort component.

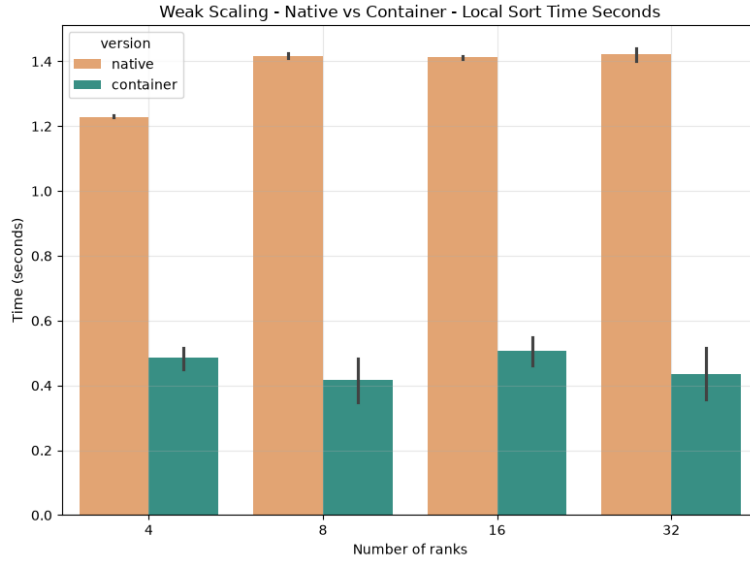


Figure 23: Local-sort execution time during weak scaling for the native and containerized implementations.

The containerized execution exhibits a lower local-sort time. However, this result should not be interpreted as evidence that the container itself accelerates the sorting algorithm. The measurements indicate that the difference originates from the execution environment and its runtime configuration rather than from an inherent advantage of containerization.

More importantly, the communication component does not show the same behavior. In fact, the communication performance of the containerized implementation is worse. This observation became particularly significant when examining the distribution workload and the MPI microbenchmarks.

5.0.4 Skewed Distribution

The same configuration of the native version was also used for the containerized one. The total execution time is shown in Figure 24.

The containerized execution shows a performance degradation compared with the native implementation. To identify the source of this degradation, the communication component was examined separately.

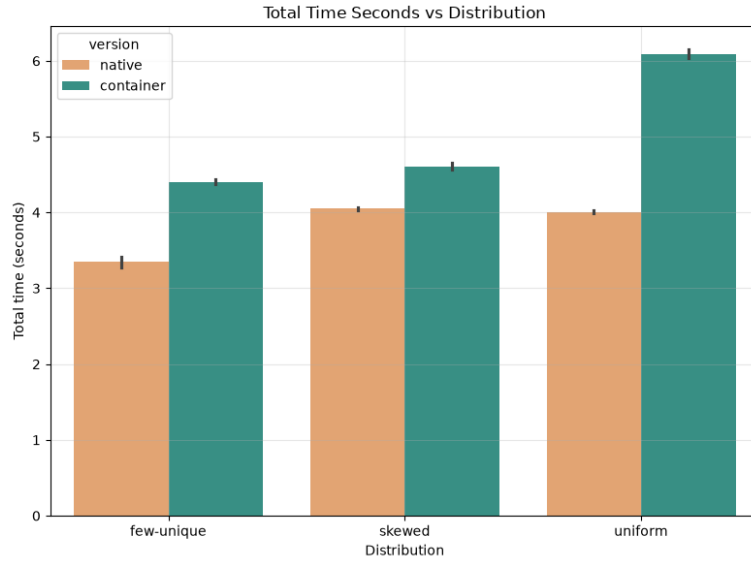


Figure 24: Total execution time of the distribution workload for the native and containerized implementations.

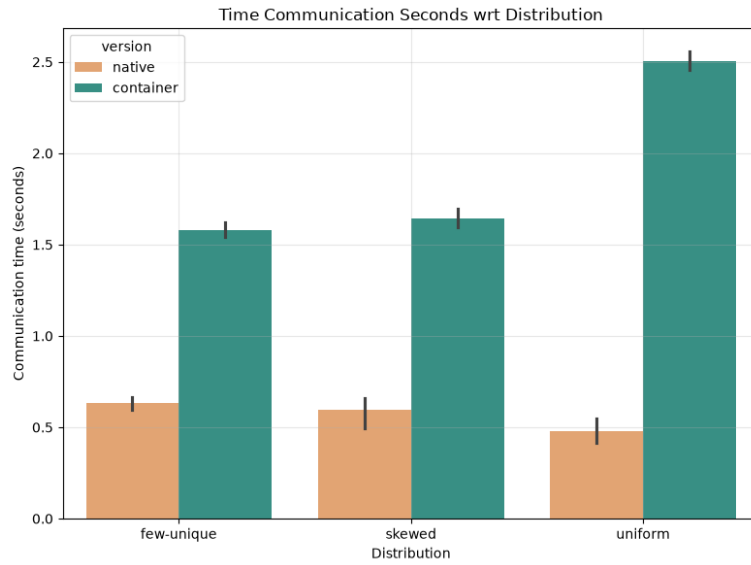


Figure 25: Communication time of the distribution workload for the native and containerized implementations.

The communication time is significantly higher in the containerized execution. This observation suggests that the main difference between the two environments is not the computational part of the application, but rather the MPI communication path used between nodes.

5.0.5 OSU Micro-Benchmarks

To isolate the communication layer from the application itself, the OSU Micro-Benchmarks were executed in both the native and containerized environments. In particular, the `MPI_Alltoallv` benchmark was selected because this collective operation is representative of the communication pattern used by the application.

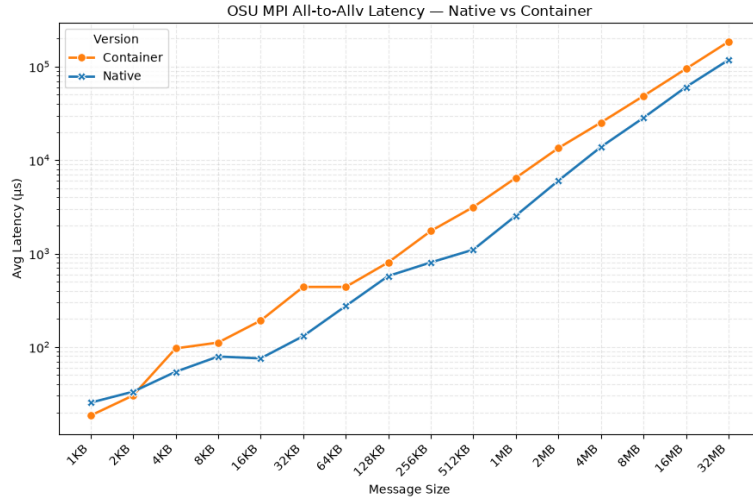


Figure 26: Native and Containerized execution of `Osu-Alltoallv` microbenchmark.

The results for the native and containerized executions are shown in Figure 26.

The microbenchmark reproduces the communication-performance difference observed in the application. The containerized execution exhibits substantially higher communication costs than the native execution. Since the benchmark isolates MPI communication from the sorting and application-specific computation, this provides strong evidence that the performance difference originates in the MPI/network stack rather than in the application itself.

6 Conclusion

Overall, the experiments showed that the implementation scales well at smaller sizes, but communication becomes the main bottleneck as the number of MPI ranks increases. In particular, the distribution phase and `MPI_Alltoallv` become increasingly important in the total execution time.

The local sorting phase is generally less problematic, especially with Radix Sort, which consistently performed well on the tested workloads.

The experiments with skewed distributions also showed that load balance can have a strong impact on both communication and merging.

Therefore, future improvements should mainly focus on reducing communication and synchronization between ranks.

One possible alternative is Histogram Sort, which could exploit the integer key space to reduce the need for sampling and global redistribution. Another interesting option is HyperQuicksort, which could limit communication by progressively partitioning subsets of processes. Personally I find HyperQuickSort the most intriguing technique, from its theoretical view point. Nonetheless I recognize the difficulty of having actual Hyper Cubes networks. I would be curious about how much the performances deteriorate working "in block". Both approaches could potentially improve scalability, especially at higher MPI rank counts. However, their actual benefits would depend on the input distribution, load balance, and communication costs.

Overall, the project suggests that further scalability is likely to come more from improving the distributed algorithm than from optimizing the local sorting itself.

Brief Clarification About the use of Generative AI tools in this project, I used Gemini Flash 3.6 to help me debugging the code and ChatGPT (free version) to help me researching some aspects and writing the report using the correct english form.

I also spent times on online forum (such as Stack Overflow), public GitHub repository and Wikipedia to find code examples, clarifications and opinions.